

5.2 Function Prototypes

In modern C programming, it is considered good practice to use prototype declarations for all functions that you call. As we mentioned, these prototypes help to ensure that the compiler can generate correct code for calling the functions, as well as allowing the compiler to catch certain mistakes you might make.

Strictly speaking, however, prototypes are optional. If you call a function for which the compiler has not seen a prototype, the compiler will do the best it can, assuming that you're calling the function correctly.

If prototypes are a good idea, and if we're going to get in the habit of writing function prototype declarations for functions we call that we've written (such as `multbytwo`), what happens for library functions such as `printf`? Where are their prototypes? The answer is in that boilerplate line

```
#include <stdio.h>
```

we've been including at the top of all of our programs. `stdio.h` is conceptually a file full of external declarations and other information pertaining to the "Standard I/O" library functions, including `printf`. The `#include` directive (which we'll meet formally in a later chapter) arranges that all of the declarations within `stdio.h` are considered by the compiler, rather as if we'd typed them all in ourselves. Somewhere within these declarations is an external function prototype declaration for `printf`, which satisfies the rule that there should be a prototype for each function we call. (For other standard library functions we call, there will be other "header files" to include.) Finally, one more thing about external function prototype declarations. We've said that the distinction between external declarations and defining instances of normal variables hinges on the presence or absence of the keyword `extern`. The situation is a little bit different for functions. The "defining instance" of a function is the function, including its body (that is, the brace-enclosed list of declarations and statements implementing the function). An external declaration of a function, even without the keyword `extern`, looks nothing like a function declaration. Therefore, the keyword `extern` is optional in function prototype declarations. If you wish, you can write

```
int multbytwo(int);
```

and this is just as good an external function prototype declaration as

```
extern int multbytwo(int);
```

(In the first form, without the `extern`, as soon as the compiler sees the semicolon, it knows it's not going to see a function body, so the declaration can't be a definition.) You may want to stay in the habit of using `extern` in all external declarations, including function declarations, since "extern = external declaration" is an easier rule to remember.

Read sequentially: [prev](#) [next](#) [up](#) [top](#)

This page by [Steve Summit](#) // [Copyright](#) 1995, 1996 // [mail feedback](#)